

Lab Tasks: GUI & Problem-Solving Agents

Task 1: Streamlit-Based Exploratory Data Analysis Interface

Develop an interactive GUI system for exploratory data analysis (EDA) using Streamlit. The system should facilitate dataset ingestion, metadata inspection, and primary visual pattern recognition, providing end-users with a structured interface for initial data exploration.

Implementation Requirements:

1. Dataset Ingestion & Metadata Inspection

- **File Upload Interface:** Implement a secure file upload module supporting CSV format. Validate that the uploaded file is correctly formatted.
- **Dataset Overview Panel:** Display key metadata including:
 - Dimensional properties (number of rows and columns)
 - Column-wise data types
 - Count of missing values per attribute
 - Basic statistical summary for numerical attributes (mean, median, min, max)

Note: Test your implementation using the titanic.csv dataset.

2. Attribute Selection & Classification

- **Dynamic Feature Selector:** Provide a dropdown to select a single column for analysis.
- **Automated Attribute Typing:** Detect whether the selected column is numerical or categorical, and update the visualization module accordingly.

3. Visualization Module

- **Conditional Rendering:**
 - **Numerical Attributes:** Generate a **histogram** depicting the value distribution. Include axis labels and a descriptive title.
 - **Categorical Attributes:** Generate a **bar chart** representing frequency counts of unique values. Include axis labels and percentage display (optional).
- Ensure that visualizations are updated dynamically upon attribute selection.

4. Interface Layout & Organization

- **Sidebar Panel:** Should have all interactive controls, including file upload and column selection.

- **Main Content Area:**

- **Top section:** Dataset preview (first 5 rows) and metadata display
- **Bottom section:** Conditional visualization canvas

Deliverables

1. Fully functional Streamlit EDA GUI meeting the above specifications.
2. Screenshots of the interface with at least one numerical and one categorical attribute plotted.

Expected Output:

Dataset Controls

Upload CSV File for Analysis

Drag and drop file here
Limit 200MB per file • CSV

Browse files

Titanic-Dataset.csv
59.8KB

Attribute Selection

Select Attribute for Visualization

Age

Exploratory Data Analysis Interface

Dataset Preview & Metadata

First 5 Rows:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22	1	0	A/5 21171	7.25	None	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Thayer)	female	38	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26	0	0	STON/O2. 3101282	7.925	None	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35	1	0	113803	53.1	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35	0	0	373450	8.05	None	S

Shape: (891, 12)

Column Data Types:

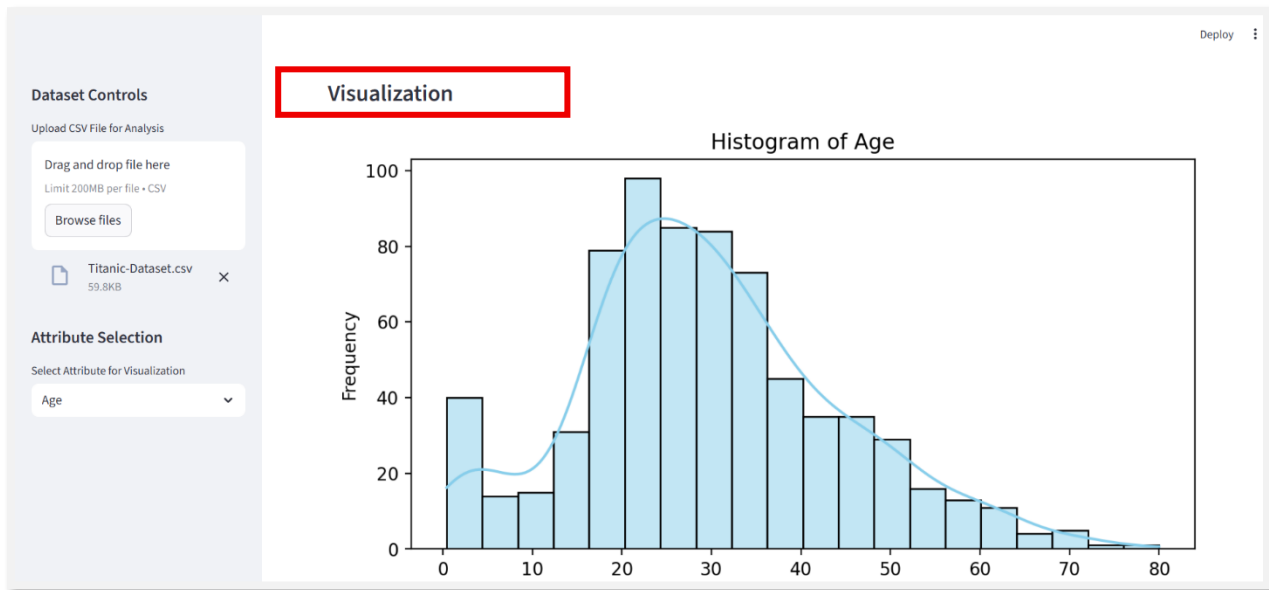
	Data Type
PassengerId	int64
Survived	int64
Pclass	int64

Missing Values per Column:

	Missing Count	Missing %
Pclass	0	0
Name	0	0
Sex	0	0
Age	177	19.8653
SibSp	0	0
Parch	0	0
Ticket	0	0
Fare	0	0
Cabin	687	77.1044
Embarked	2	0.2245

Basic Numerical Statistics:

	count	mean	std	min	25%	50%	75%	max
PassengerId	891	446	257.3538	1	223.5	446	668.5	891
Survived	891	0.3838	0.4866	0	0	0	1	1
Pclass	891	2.3086	0.8361	1	2	2	2	2



Task 2: Route Planning Agent for Delivery Service

Objective

Implement a problem-solving agent using an atomic representation to determine an optimal delivery route for a logistics company operating in a simplified urban network.

Scenario

You are to develop a **Route Planning Agent** for a delivery service operating in a fictional city district represented as a graph of interconnected locations (nodes) and roads (edges). The agent must compute a fixed sequence of movements to deliver a package from a **depot** to a **customer location** without relying on real-time traffic updates (future percepts).

Specifications

1. Goal Formulation

- Define the goal state as the *customer's location*.
- Performance measure: *Minimize total travel distance*.

2. Problem Formulation

- **Initial State:** Depot location (node ID).
- **Possible Actions:** MoveTo(Location) where Location is an adjacent node.
- **Transition Model:** RESULT(State, MoveTo(Location)) returns the new state (Location).
- **Goal Test:** Check if current state equals customer location.
- **Path Cost:** Sum of edge weights (distances) between consecutive nodes.

3. Implementation Requirements

- Represent the city graph using an adjacency list.

Example:

```
citygraph = {  
    'A': [('B', 4), ('C', 3)],  
    'B': [('D', 5), ('E', 6)],  
}
```

- Implement a **state-space graph** generator that enumerates all reachable states from the initial state.
- Define the five problem components programmatically. (i.e. **Initial State, Actions, Transition Model, Goal Test, Path Cost**)
- Output the **solution sequence** (list of actions) and its total cost.

4. Constraints

- Assume a static environment (no dynamic obstacles).
- Use a pre-defined graph with at least 5 nodes and weighted edges.
- The agent must not revisit nodes to avoid cycles. (*i.e. Do not allow the next state to be any state that already appears in the current path.*)

Printed output showing:

- Initial state and goal state.
- State-space graph (as a list of reachable states).
- Solution action sequence and total path cost.

Task 3: Puzzle-Solving Agent for 8-Tile Sliding Puzzle

Objective

Implement a problem-solving agent to solve the 8-puzzle using atomic representation and state-space search, without using advanced search algorithms.

Scenario

The 8-puzzle consists of a 3×3 grid with 8 numbered tiles and one blank space. The agent must determine a sequence of tile slides (blank movements) to reach a goal configuration from a given initial configuration.

Specifications

1. Goal Formulation

- Goal state: Tiles in ascending order with blank in bottom-right.

2. Problem Formulation

- **Initial State:** 3×3 matrix representing tile arrangement.
- **Possible Actions:** Move blank Up, Down, Left, Right (if applicable).
- **Transition Model:** Returns new puzzle state after moving blank.
- **Goal Test:** Check if current state matches goal configuration.
- **Path Cost:** Each move costs 1.

3. Implementation Requirements

- Represent puzzle state as a tuple of tuples for hashability.
- Generate all valid successor states for a given state.
- Build a state-space graph up to a certain depth (e.g., depth 10).
- Output the sequence of moves if a solution is found within depth limit.

4. Constraints

- Do not use BFS, DFS, or heuristic search, only generate state space.
- Limit state-space exploration to avoid infinite loops.
- Print Output showing:
 - Number of states generated.
 - Solution path (if found).
 - Path cost.

Task 4: Robotic Arm Configuration Planner

Objective

Design and implement a problem-solving agent for a simple 2D robotic arm to achieve a target end-effector position through a sequence of joint movements.

Scenario

A robotic arm with two rotational joints operates in a 2D plane. The agent must compute a fixed sequence of joint angle adjustments to move the end-effector from an initial configuration to a target position, avoiding self-collision and joint limit violations.

Specifications

1. Goal Formulation

- Goal: End-effector position within a tolerance (e.g., Euclidean distance < 0.1 units).
- Performance measure: Minimize number of movements and energy expenditure.

2. Problem Formulation

- **Initial State:** Tuple of joint angles (θ_1, θ_2) .
- **Possible Actions:** `RotateJoint(JointID,  $\Delta\theta$ )` where $\Delta\theta \in \{-10^\circ, 0^\circ, +10^\circ\}$.
- **Transition Model:** Apply angle change and compute new end-effector position via forward kinematics.
- **Goal Test:** Check if end-effector position is within tolerance of target.
`math.dist(pos, target) < TOLERANCE`
- **Path Cost:** Each action costs 1 unit; penalize large $\Delta\theta$ if desired.

3. Implementation Requirements

- Implement forward kinematics for a 2-link arm.

```
t1 = math.radians(theta1)
t2 = math.radians(theta2)
x = L1 * math.cos(t1) + L2 * math.cos(t1 + t2)
y = L1 * math.sin(t1) + L2 * math.sin(t1 + t2)
```
- Generate the state-space graph for a bounded joint angle range (e.g., -90° to 90°).
- Include joint limit constraints in transition model.
- Output the sequence of joint movements and final end-effector position.

4. Constraints

- Discrete action space (no continuous adjustments).
 - Avoid states where links intersect (optional collision detection).
- Visualize initial, intermediate, and final arm configurations (using matplotlib).

- Show solution path and total cost.

Submission Guidelines:

1. Prepare a **Word report** containing screenshots of the output for **all tasks**.
2. Ensure that each screenshot is **clearly placed under its respective task**.
3. **Download the .ipynb file** from Jupyter Notebook.
4. **Upload the Word report, .ipynb, and .py files to GCR. Do NOT upload a ZIP file.**
5. **Failure to follow the submission guidelines will result in a deduction of 50% of the total marks.**